remove, and update fields. If you decide to use a custom serializer, you need several SerDes; depending on the version of the event, the right one will be selected.

If a new field is added when loading the data into the persistent actor, a default value must be added to the field. This is an alternative to updating all previous events, as mentioned earlier. A third value, such as "no data," could be added to the old event values when the event is loaded—but this would require using the Akka Persistence event adapters, which is beyond the scope of this chapter (see http://mng.bz/7De7).

9.4 Running example

The `Main` application in the book's Git repository in the chapter09b project uses what you saw in the previous section. This application adds tagging, snapshotting, and monitoring to the `SPContainer` as follows:

```
object SPContainer {
     ...

      .withTagger {
        case _ => Set("container-tag-" + containerId.toInt % 3)
      }
      .withRetention(RetentionCriteria
        .snapshotEvery(numberOfEvents = 100, keepNSnapshots = 2))
      .onPersistFailure(
        SupervisorStrategy.restartWithBackoff(
          minBackoff = 10.seconds,
          maxBackoff = 60.seconds,
          randomFactor = 0.1))
     ...
}
```

You should always add these to any application. With the preceding configuration, the application takes a snapshot after every 100 events. If the actor crashes, it tries to restart after 10 seconds with a random accuracy of 10% and with an increasing delay of 20, 30, 40, and 50 seconds, up to a maximum of 60 seconds. With the tags, all events are divided into three categories: `container-tag-0`, `container-tag-1`, and `container-tag-2`. This way, three consumers can use the journal in parallel.

The chapter09b project also has a database that uses three tables—`event_journal`, `event_tags`, and `snapshots`—to store the events in the journal, along with the application's configuration. The code sets up a Postgres database via `docker-compose up` and provides the SQL scripts to create the tables for journal events, tags, and snapshots. All the details are in README.md. Once the database is running and the three tables are created, you can start the application by typing `sbt chapter09b/run` at the command line.

When the application starts, you can interact with the cluster through the sbt shell by writing in it the message you want to send to the container. To send a message, enter values for